

JPA

Spring Data

- The traditional data access problem
 - Every interaction with a data store requires boilerplate
 - *Obtaining a connection*
 - *Preparing a statement*
 - *Executing the statement*
 - *Mapping results*
 - *Handling exceptions*
 - *Releasing resources*
 - Boilerplate
 - *Repetitive, error-prone code*
 - *Clutters the application with infrastructure concerns that have nothing to do with business logic*

```
// Raw JDBC -- every operation requires this scaffolding
public Employee findById(Long id) {
    Connection conn = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        conn = dataSource.getConnection();
        stmt = conn.prepareStatement(
            "SELECT employee_id, full_name, salary FROM employees WHERE employee_id = ?"
        );
        stmt.setLong(1, id);
        rs = stmt.executeQuery();
        if (rs.next()) {
            return new Employee(
                rs.getLong("employee_id"),
                rs.getString("full_name"),
                rs.getBigDecimal("salary")
            );
        }
        return null;
    } catch (SQLException e) {
        throw new RuntimeException("Database error", e);
    } finally {
        // Must close in reverse order -- if any close throws, others are skipped
        if (rs != null) try { rs.close(); } catch (SQLException ignored) {}
        if (stmt != null) try { stmt.close(); } catch (SQLException ignored) {}
        if (conn != null) try { conn.close(); } catch (SQLException ignored) {}
    }
}
```

Spring Data

- The traditional data access problem
 - Different data stores each have their own APIs, connection models, and error handling conventions
 - *Relational databases*
 - *Key-value stores*
 - *Document stores*
 - *Directory services*
 - This means the application code is tightly coupled to the specific data store technology
 - And also tightly coupled to the underlying data model
 - The resulting system is very brittle

```
// Raw JDBC -- every operation requires this scaffolding
public Employee findById(Long id) {
    Connection conn = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        conn = dataSource.getConnection();
        stmt = conn.prepareStatement(
            "SELECT employee_id, full_name, salary FROM employees WHERE employee_id = ?"
        );
        stmt.setLong(1, id);
        rs = stmt.executeQuery();
        if (rs.next()) {
            return new Employee(
                rs.getLong("employee_id"),
                rs.getString("full_name"),
                rs.getBigDecimal("salary")
            );
        }
        return null;
    } catch (SQLException e) {
        throw new RuntimeException("Database error", e);
    } finally {
        // Must close in reverse order -- if any close throws, others are skipped
        if (rs != null) try { rs.close(); } catch (SQLException ignored) {}
        if (stmt != null) try { stmt.close(); } catch (SQLException ignored) {}
        if (conn != null) try { conn.close(); } catch (SQLException ignored) {}
    }
}
```

Spring Data

- Design goal of Spring Data
 - Decouple the boilerplate scaffolding from the application code
 - *Declarative statement of the interface to the repository*
 - *Let the framework generate the boilerplate code*
 - Data access operations follow predictable patterns
 - *Find by ID, find all, save, delete, find by criteria*
 - *These patterns are the same regardless of whether the store is Oracle, Redis, or LDAP*
 - *Spring Data codifies these patterns into reusable abstractions so developers declare what they need rather than implementing it each time*

```
// The entire above method is replaced by this single interface declaration
public interface EmployeeRepository extends CrudRepository<Employee, Long> {
    // findById(Long id) is already provided by CrudRepository
}
```

Design Goals

- 1: Reduce boilerplate to the minimum necessary expression of intent
 - The developer declares what data operations the application needs
 - The framework provides implementations of those operations
 - The developer does not implement what the framework already knows how to do
- 2: Provide a consistent programming model across different data stores
 - The same repository abstractions, the same annotation model, and the same query derivation mechanism work whether the underlying store is a relational database, a key-value store, a document store, or a directory service
 - Switching from one store to another changes configuration and dependencies but not the application's data access logic

Design Goals

- 3: Support store-specific features without hiding them
 - Spring Data does not force a lowest-common-denominator abstraction that loses the power of specific stores
 - *Oracle-specific SQL can be expressed with @Query; Redis pipeline operations are available; LDAP filter syntax is supported*
 - *The abstraction reduces boilerplate but never prevents access to the full capability of the underlying store*
- 4: Integrate naturally with the Spring ecosystem
 - Repositories are Spring beans
 - *They participate in dependency injection, transaction management, and the application context lifecycle*
 - Spring Data works with Spring's transaction manager so that data access operations participate in the same transactions as service layer logic
 - Configuration follows Spring Boot's auto-configuration model
 - *Adding a dependency and providing connection properties is usually sufficient to get a working data access layer*

Repository Pattern

- The Repository pattern
 - Described by Eric Evans in Domain-Driven Design (2003)
- A repository
 - An object that mediates between the domain layer and the data mapping layer
 - Presents the application with a collection-like interface for accessing domain objects
 - The application asks the repository for objects as if they were in a collection in memory
 - The repository handles the mechanics of retrieval from the actual store

```
Repository<T, ID>                -- marker interface, no methods
└─ CrudRepository<T, ID>         -- save, findById, findAll, delete, count, exists
    └─ PagingAndSortingRepository<T, ID> -- findAll with Sort and Pageable
        └─ JpaRepository<T, ID>      -- JPA-specific: flush, saveAndFlush, batch operations
```


Repository Pattern

- Declaring a repository
 - T is the domain type (the entity class)
 - ID is the type of the entity's primary key
 - Each level adds capabilities to the one above it
 - The application extends the interface appropriate for its needs
 - *It does not implement anything*

```
Repository<T, ID>                -- marker interface, no methods
└─ CrudRepository<T, ID>        -- save, findById, findAll, delete, count, exists
    └─ PagingAndSortingRepository<T, ID> -- findAll with Sort and Pageable
        └─ JpaRepository<T, ID>      -- JPA-specific: flush, saveAndFlush, batch operations
```

```
// Extend the appropriate interface for the required capability level
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Spring Data derives a query from the method name automatically
    List<Employee> findByDepartmentIdAndStatus(Long departmentId, String status);

    // Spring Data derives an ORDER BY from the method name
    List<Employee> findByDepartmentIdOrderBySalaryDesc(Long departmentId);

    // For more complex queries, provide the JPQL explicitly
    @Query("SELECT e FROM Employee e WHERE e.salary > :threshold AND e.status = 'ACTIVE'")
    List<Employee> findHighEarners(@Param("threshold") BigDecimal threshold);
}
```


Repository Pattern

- Spring Data
 - Provides the implementation
 - At application startup, Spring Data scans for interfaces that extend Repository
 - For each one, it generates a proxy implementation using Java reflection and the JDK proxy mechanism
 - The proxy implements every method: those inherited from the parent interface and those declared in the application interface
 - The proxy is registered as a Spring bean and is available for injection throughout the application

```
Repository<T, ID>                -- marker interface, no methods
└─ CrudRepository<T, ID>         -- save, findById, findAll, delete, count, exists
    └─ PagingAndSortingRepository<T, ID> -- findAll with Sort and Pageable
        └─ JpaRepository<T, ID>      -- JPA-specific: flush, saveAndFlush, batch operations
```

```
// Extend the appropriate interface for the required capability level
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Spring Data derives a query from the method name automatically
    List<Employee> findByDepartmentIdAndStatus(Long departmentId, String status);

    // Spring Data derives an ORDER BY from the method name
    List<Employee> findByDepartmentIdOrderBySalaryDesc(Long departmentId);

    // For more complex queries, provide the JPQL explicitly
    @Query("SELECT e FROM Employee e WHERE e.salary > :threshold AND e.status = 'ACTIVE'")
    List<Employee> findHighEarners(@Param("threshold") BigDecimal threshold);
}
```

Repository Pattern

- What the JDK proxy mechanism is
 - Facility in the Java standard library
 - *Creates a concrete object implementing one or more interfaces at runtime without any hand-written class*
 - *The created object is the proxy.*
 - Two things are required to create a JDK proxy
 - *The interface or interfaces it should implement*
 - *An [InvocationHandler](#), a single method that receives every method call made on the proxy and decides what to do with it*
 - A proxy is an object that stands in for another object and intercepts calls made to it

```
// This is conceptually what Spring Data does internally
// (simplified -- the real implementation is considerably more sophisticated)

InvocationHandler handler = (proxy, method, args) -> {
    // Every call to any method on the repository arrives here
    // method.getName() tells us which method was called
    // args contains the arguments

    if (method.getName().equals("findById")) {
        // Generate and execute: SELECT * FROM employees WHERE employee_id = ?
        // Map the ResultSet to an Employee object
        // Return it
    }

    if (method.getName().startsWith("findBy")) {
        // Parse the rest of the method name to derive the WHERE clause
        // Generate and execute the appropriate SQL
        // Map the results
    }

    if (method.isAnnotationPresent(Query.class)) {
        // Read the JPQL or SQL from the @Query annotation
        // Bind the parameters
        // Execute and map the results
    }
    // ... and so on for every supported pattern
};

Object repositoryProxy = Proxy.newProxyInstance(
    classLoader,
    new Class[]{ EmployeeRepository.class },
    handler
);
```

Repository Pattern

- In Spring Data
 - The developer declares a repository interface but writes no implementation class
 - At application startup, Spring Data needs something concrete that can be injected wherever that interface is used
 - The proxy is that concrete object created by Spring

```
// This is conceptually what Spring Data does internally
// (simplified -- the real implementation is considerably more sophisticated)

InvocationHandler handler = (proxy, method, args) -> {
    // Every call to any method on the repository arrives here
    // method.getName() tells us which method was called
    // args contains the arguments

    if (method.getName().equals("findById")) {
        // Generate and execute: SELECT * FROM employees WHERE employee_id = ?
        // Map the ResultSet to an Employee object
        // Return it
    }

    if (method.getName().startsWith("findBy")) {
        // Parse the rest of the method name to derive the WHERE clause
        // Generate and execute the appropriate SQL
        // Map the results
    }

    if (method.isAnnotationPresent(Query.class)) {
        // Read the JPQL or SQL from the @Query annotation
        // Bind the parameters
        // Execute and map the results
    }

    // ... and so on for every supported pattern
};

Object repositoryProxy = Proxy.newProxyInstance(
    classLoader,
    new Class[]{ EmployeeRepository.class },
    handler
);
```

Repository Pattern

- Java reflection
 - Capabilities built into the Java runtime that allows code to inspect and interact with the structure of other code at runtime rather than at compile time
 - Through reflection, a running program can
 - *Discover what methods an interface declares, without knowing the interface at compile time*
 - *Read annotations on those methods and their parameters*
 - *Create new objects and invoke methods dynamically, by name, without a direct call in the source code*

```
// This is conceptually what Spring Data does internally
// (simplified -- the real implementation is considerably more sophisticated)

InvocationHandler handler = (proxy, method, args) -> {
    // Every call to any method on the repository arrives here
    // method.getName() tells us which method was called
    // args contains the arguments

    if (method.getName().equals("findById")) {
        // Generate and execute: SELECT * FROM employees WHERE employee_id = ?
        // Map the ResultSet to an Employee object
        // Return it
    }

    if (method.getName().startsWith("findBy")) {
        // Parse the rest of the method name to derive the WHERE clause
        // Generate and execute the appropriate SQL
        // Map the results
    }

    if (method.isAnnotationPresent(Query.class)) {
        // Read the JPQL or SQL from the @Query annotation
        // Bind the parameters
        // Execute and map the results
    }
    // ... and so on for every supported pattern
};

Object repositoryProxy = Proxy.newProxyInstance(
    classLoader,
    new Class[]{ EmployeeRepository.class },
    handler
);
```

Repository Pattern

- Java reflection
 - Spring Data uses reflection to inspect the repository interface the developer declared
 - Reads the interface name, the methods declared on it, the parent interfaces it extends, and all the annotations on those methods
 - This inspection happens at startup and gives Spring Data everything it needs to understand what the repository is supposed to do.

```
// This is conceptually what Spring Data does internally
// (simplified -- the real implementation is considerably more sophisticated)

InvocationHandler handler = (proxy, method, args) -> {
    // Every call to any method on the repository arrives here
    // method.getName() tells us which method was called
    // args contains the arguments

    if (method.getName().equals("findById")) {
        // Generate and execute: SELECT * FROM employees WHERE employee_id = ?
        // Map the ResultSet to an Employee object
        // Return it
    }

    if (method.getName().startsWith("findBy")) {
        // Parse the rest of the method name to derive the WHERE clause
        // Generate and execute the appropriate SQL
        // Map the results
    }

    if (method.isAnnotationPresent(Query.class)) {
        // Read the JPQL or SQL from the @Query annotation
        // Bind the parameters
        // Execute and map the results
    }
    // ... and so on for every supported pattern
};

Object repositoryProxy = Proxy.newProxyInstance(
    classLoader,
    new Class[]{ EmployeeRepository.class },
    handler
);
```

Repository Pattern

- How Spring Data works
 - At application startup, Spring Data scans for interfaces that extend Repository
 - For each one, it generates a proxy implementation using Java reflection and the JDK proxy mechanism
 - The proxy implements every method
 - *Those inherited from the parent interface and those declared in the application interface*
 - The proxy is registered as a Spring bean and is available for injection throughout the application

```
// This is conceptually what Spring Data does internally
// (simplified -- the real implementation is considerably more sophisticated)

InvocationHandler handler = (proxy, method, args) -> {
    // Every call to any method on the repository arrives here
    // method.getName() tells us which method was called
    // args contains the arguments

    if (method.getName().equals("findById")) {
        // Generate and execute: SELECT * FROM employees WHERE employee_id = ?
        // Map the ResultSet to an Employee object
        // Return it
    }

    if (method.getName().startsWith("findBy")) {
        // Parse the rest of the method name to derive the WHERE clause
        // Generate and execute the appropriate SQL
        // Map the results
    }

    if (method.isAnnotationPresent(Query.class)) {
        // Read the JPQL or SQL from the @Query annotation
        // Bind the parameters
        // Execute and map the results
    }
    // ... and so on for every supported pattern
};

Object repositoryProxy = Proxy.newProxyInstance(
    classLoader,
    new Class[]{ EmployeeRepository.class },
    handler
);
```


Repository Pattern

- **CrudRepository**

- Sufficient for most operations
- **PagingAndSortingRepository**
 - *Adds the ability to retrieve data in pages and in a specified order without writing any pagination logic*
- **JpaRepository**
 - *Adds JPA-specific operations that are not in the generic interfaces*
 - *An application that only needs basic CRUD should extend **CrudRepository** there is no reason to pull in JPA-specific capabilities if they are not needed*

```
Repository<T, ID>                -- marker interface, no methods
└─ CrudRepository<T, ID>         -- save, findById, findAll, delete, count, exists
    └─ PagingAndSortingRepository<T, ID> -- findAll with Sort and Pageable
        └─ JpaRepository<T, ID>      -- JPA-specific: flush, saveAndFlush, batch operations
```

```
// Extend the appropriate interface for the required capability level
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Spring Data derives a query from the method name automatically
    List<Employee> findByDepartmentIdAndStatus(Long departmentId, String status);

    // Spring Data derives an ORDER BY from the method name
    List<Employee> findByDepartmentIdOrderBySalaryDesc(Long departmentId);

    // For more complex queries, provide the JPQL explicitly
    @Query("SELECT e FROM Employee e WHERE e.salary > :threshold AND e.status = 'ACTIVE'")
    List<Employee> findHighEarners(@Param("threshold") BigDecimal threshold);
}
```


Repository Pattern

- Spring Data
 - Parses the method name and constructs a query from it
 - *findByDepartmentIdAndStatus* becomes a query with a *WHERE* clause on *department_id* and *status*
 - *findByDepartmentIdOrderBySalaryDesc* adds an *ORDER BY* salary *DESC*
 - Works well for simple queries and eliminates the need to write boilerplate query strings for common lookup patterns
 - For complex queries, the `@Query` annotation provides an escape hatch
 - *Developer writes the query explicitly and Spring Data handles binding parameters and mapping results*

```
Repository<T, ID>                -- marker interface, no methods
└─ CrudRepository<T, ID>         -- save, findById, findAll, delete, count, exists
    └─ PagingAndSortingRepository<T, ID> -- findAll with Sort and Pageable
        └─ JpaRepository<T, ID>      -- JPA-specific: flush, saveAndFlush, batch operations
```

```
// Extend the appropriate interface for the required capability level
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Spring Data derives a query from the method name automatically
    List<Employee> findByDepartmentIdAndStatus(Long departmentId, String status);

    // Spring Data derives an ORDER BY from the method name
    List<Employee> findByDepartmentIdOrderBySalaryDesc(Long departmentId);

    // For more complex queries, provide the JPQL explicitly
    @Query("SELECT e FROM Employee e WHERE e.salary > :threshold AND e.status = 'ACTIVE'")
    List<Employee> findHighEarners(@Param("threshold") BigDecimal threshold);
}
```

Query Derivation

- Turning method names into queries
 - Spring Data parses repository method names according to a grammar
 - The method name is split into
 - *A subject or what to do*
 - *And a predicate or the conditions*
 - Spring Data constructs the appropriate query for the underlying store from this parsed structure
 - This is called method name derivation

Method name grammar

```
find...By...    -- SELECT query
count...By...   -- COUNT query
exists...By...  -- EXISTS check returning boolean
delete...By...  -- DELETE statement
```

Conditions:

```
And             -- AND between two conditions
Or              -- OR between two conditions
Between         -- BETWEEN lower AND upper
LessThan        -- < value
GreaterThan     -- > value
Like            -- LIKE pattern
NotLike         -- NOT LIKE pattern
In              -- IN (collection)
NotIn           -- NOT IN (collection)
IsNull          -- IS NULL
IsNotNull       -- IS NOT NULL
OrderBy         -- ORDER BY column Asc/Desc
```

Query Derivation

- Method name derivation
 - Powerful feature for simple lookups
 - *Not a replacement for SQL or JPQL for anything beyond straightforward single-table queries*
 - *Natural fit is simple find-by-field operations*
 - *More complex queries should use explicit `@Query`*
 - Rules of thumb
 - *If the method name requires reading twice to understand what query it represents, replace it with `@Query` with a clear name and an explicit query string*
 - *Method names become unreadable beyond two or three conditions*
 - *Any query involving joins, subqueries, aggregations, or complex expressions requires `@Query`*

```
// WHERE status = ? AND department_id = ?
List<Employee> findByStatusAndDepartmentId(String status, Long departmentId);

// WHERE salary BETWEEN ? AND ?
List<Employee> findBySalaryBetween(BigDecimal min, BigDecimal max);

// WHERE full_name LIKE ? (caller must supply the % characters)
List<Employee> findByFullNameLike(String pattern);

// WHERE department_id IN (?, ?, ...)
List<Employee> findByDepartmentIdIn(Collection<Long> departmentIds);

// WHERE status IS NOT NULL ORDER BY hire_date DESC
List<Employee> findByStatusIsNotNullOrderByHireDateDesc();

// Returns true if any matching row exists
boolean existsByEmailAddress(String email);

// Returns the count of matching rows
long countByStatus(String status);
```

Query Derivation

- Method name derivation
 - The Like condition
 - *Spring Data does not add the % wildcard characters automatically*
 - *The caller must supply the pattern with % characters included*
 - The Contains variant that wraps the value in % automatically
 - *StartingWith and EndingWith used for prefix and suffix matches*
 - *These are more readable than Like for the common cases*
 - *FindByFullNameContaining has LIKE functionality*

```
// WHERE status = ? AND department_id = ?
List<Employee> findByStatusAndDepartmentId(String status, Long departmentId);

// WHERE salary BETWEEN ? AND ?
List<Employee> findBySalaryBetween(BigDecimal min, BigDecimal max);

// WHERE full_name LIKE ? (caller must supply the % characters)
List<Employee> findByFullNameLike(String pattern);

// WHERE department_id IN (?, ?, ...)
List<Employee> findByDepartmentIdIn(Collection<Long> departmentIds);

// WHERE status IS NOT NULL ORDER BY hire_date DESC
List<Employee> findByStatusIsNotNullOrderByHireDateDesc();

// Returns true if any matching row exists
boolean existsByEmailAddress(String email);

// Returns the count of matching rows
long countByStatus(String status);
```

Entity Mapping

- In Spring Data
 - A Java class is the entity
 - *Represents a table row, a document, a cache entry, or a directory object*
 - Annotations on the class and its fields tell Spring Data how to map between the Java object and the underlying store
 - The choice of annotations depends on the Spring Data module being used

```
@Entity // marks this class as a JPA-managed entity
@Table(name = "employees") // maps to the employees table
public class Employee {

    @Id // marks this field as the primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // uses the DB identity column
    @Column(name = "employee_id")
    private Long employeeId;

    @Column(name = "full_name", nullable = false, length = 200)
    private String fullName;

    @Column(name = "salary", precision = 15, scale = 2)
    private BigDecimal salary;

    @Column(name = "hire_date")
    private LocalDate hireDate;

    @ManyToOne(fetch = FetchType.LAZY) // lazy: do not load department unless accessed
    @JoinColumn(name = "department_id")
    private Department department;

    // getters and setters omitted
}
```

JDBC Mapping

- JPA
 - Uses `@Entity` and supports bi-directional relationships, lazy loading, and a managed entity lifecycle
- Spring Data JDBC
 - Uses `@Table` only, has no lazy loading, and models relationships through aggregate roots rather than navigable object references
 - These are not just syntax differences
 - *They reflect fundamentally different approaches to the object-relational mapping problem*

```
@Table("employees")           // maps to the employees table
public class Employee {

    @Id
    private Long employeeId;      // mapped to employee_id by naming convention

    private String fullName;      // mapped to full_name by naming convention
    private BigDecimal salary;
    private LocalDate hireDate;

    // No relationship annotations -- Spring Data JDBC uses aggregate roots
    // instead of bi-directional object graph navigation
}
```


JDBC Mapping

- Spring Data JDBC
 - Lightweight relational data access module that maps Java objects to relational tables without a full ORM layer
 - Does not use JPA or Hibernate
 - *It has its own mapping layer built on Spring's [JdbcTemplate](#)*
 - It follows the Aggregate Root pattern from Domain-Driven Design
 - *Each repository manages one aggregate*
 - *Objects within the aggregate are loaded and saved together as a unit*

```
// JPA approach -- navigable object reference
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "department_id")
private Department department; // holds a Department object (or a proxy for one)

// Usage: traverses the object graph, may trigger a database query silently
String deptName = employee.getDepartment().getDepartmentName();

// Spring Data JDBC approach -- aggregate boundary, ID reference only
@Column("department_id")
private Long departmentId; // holds an ID, not an object

// Usage: explicit repository call -- the database access is visible
Department dept = departmentRepository.findById(employee.getDepartmentId())
    .orElseThrow();
String deptName = dept.getDepartmentName();
```

JDBC Mapping

- The aggregate root model
 - An aggregate is a cluster of objects that belong together and are treated as a unit for data access purposes
 - The aggregate root is the top-level object in that cluster
 - *The repository operates on the root, not on individual parts*
 - When the root is saved, everything inside it is saved
 - When it is loaded, everything inside it is loaded
 - There is no lazy loading
 - *If the data is part of the aggregate, it is always loaded with it*

```
// Employee is the aggregate root
// Address is part of the Employee aggregate
@Table("employees")
public class Employee {

    @Id
    private Long employeeId;
    private String fullName;
    private BigDecimal salary;

    @Embedded(onEmpty = Embedded.OnEmpty.USE_NULL)
    private Address address; // stored in the employees table, not a separate entity
}

// Repository operates on the aggregate root
public interface EmployeeRepository extends CrudRepository<Employee, Long> {
    List<Employee> findByAddressCity(String city);
}
```

JDBC Mapping

- When Spring Data JDBC is the right choice
 - The domain model is relatively simple and maps cleanly to a relational schema
 - The application needs predictable, explicit SQL behaviour without ORM magic
 - Performance is important and the overhead of JPA's managed entity lifecycle is not justified
 - The team values simplicity and explicit control over convenience and automation

```
// Employee is the aggregate root
// Address is part of the Employee aggregate
@Table("employees")
public class Employee {

    @Id
    private Long employeeId;
    private String fullName;
    private BigDecimal salary;

    @Embedded(onEmpty = Embedded.OnEmpty.USE_NULL)
    private Address address; // stored in the employees table, not a separate entity
}

// Repository operates on the aggregate root
public interface EmployeeRepository extends CrudRepository<Employee, Long> {
    List<Employee> findByAddressCity(String city);
}
```

Spring Data JPA

- Spring Data JPA
 - The most feature-rich relational data access module in Spring Data
 - Provides the Spring Data repository abstraction on top of the Java Persistence API (JPA)
 - *Using Hibernate as the default JPA provider*
 - Manages the full lifecycle of entity objects: the application works with Java objects and JPA translates those operations to SQL automatically

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {  
  
    // JPQL query -- operates on entity class names and field names, not table/column names  
    @Query("SELECT e FROM Employee e JOIN FETCH e.department " +  
           "WHERE e.department.id = :deptId AND e.status = :status")  
    List<Employee> findWithDepartmentByDeptIdAndStatus(  
        @Param("deptId") Long deptId,  
        @Param("status") String status  
    );  
  
    // Native SQL query for Oracle-specific syntax  
    @Query(value = "SELECT * FROM employees WHERE ROWNUM <= :limit " +  
           "ORDER BY salary DESC",  
           nativeQuery = true)  
    List<Employee> findTopEarnings(@Param("limit") int limit);  
  
    // Derived query with pagination -- Spring Data handles the OFFSET/FETCH automatically  
    Page<Employee> findByStatus(String status, Pageable pageable);  
}
```

- The JPA entity lifecycle
 - Transient
 - *The object exists in Java but is not associated with any database row*
 - Persistent (managed)
 - *The object is associated with a database row and tracked by the JPA session (EntityManager)*
 - *Any change to the object is automatically detected and written to the database at the end of the transaction*
 - Detached
 - *The object was previously managed but the session has closed -- changes are not tracked*
 - Removed
 - *The object is marked for deletion and will be deleted at the end of the transaction*

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {  
  
    // JPQL query -- operates on entity class names and field names, not table/column names  
    @Query("SELECT e FROM Employee e JOIN FETCH e.department " +  
           "WHERE e.department.id = :deptId AND e.status = :status")  
    List<Employee> findWithDepartmentByDeptIdAndStatus(  
        @Param("deptId") Long deptId,  
        @Param("status") String status  
    );  
  
    // Native SQL query for Oracle-specific syntax  
    @Query(value = "SELECT * FROM employees WHERE ROWNUM <= :limit " +  
                  "ORDER BY salary DESC",  
           nativeQuery = true)  
    List<Employee> findTopEarnings(@Param("limit") int limit);  
  
    // Derived query with pagination -- Spring Data handles the OFFSET/FETCH automatically  
    Page<Employee> findByStatus(String status, Pageable pageable);  
}
```

Spring Data JPA

- What JPA provides over Spring Data JDBC
 - Lazy loading
 - *Related objects are not loaded from the database until they are accessed*
 - Dirty checking
 - *JPA detects changes to managed entities and generates UPDATE statements automatically without explicit save calls*
 - First-level cache
 - *Within a session, JPA returns the same object instance for the same primary key, reducing database reads*
 - JPQL
 - *An object-oriented query language that operates on entity classes rather than tables*
 - Cascading
 - *Operations on a parent entity can automatically cascade to child entities*

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {  
  
    // JPQL query -- operates on entity class names and field names, not table/column names  
    @Query("SELECT e FROM Employee e JOIN FETCH e.department " +  
           "WHERE e.department.id = :deptId AND e.status = :status")  
    List<Employee> findWithDepartmentByDeptIdAndStatus(  
        @Param("deptId") Long deptId,  
        @Param("status") String status  
    );  
  
    // Native SQL query for Oracle-specific syntax  
    @Query(value = "SELECT * FROM employees WHERE ROWNUM <= :limit " +  
                  "ORDER BY salary DESC",  
           nativeQuery = true)  
    List<Employee> findTopEarners(@Param("limit") int limit);  
  
    // Derived query with pagination -- Spring Data handles the OFFSET/FETCH automatically  
    Page<Employee> findByStatus(String status, Pageable pageable);  
}
```


Comparison

- Fundamental differences
 - Spring Data JDBC follows the principle that
 - *SQL is the right language for relational data*
 - *The framework's job is to reduce the boilerplate of executing it*
 - Spring Data JPA follows the principle that
 - *The application should work with objects*
 - *The framework should handle the translation to SQL automatically*

Dimension	Spring Data JDBC	Spring Data JPA
Underlying technology	Spring JdbcTemplate -- no ORM	JPA (Hibernate) -- full ORM
Entity lifecycle	None -- objects are plain Java objects	Managed -- JPA tracks changes to persistent entities
Lazy loading	Not supported -- load the whole aggregate	Supported -- related objects loaded on access
Dirty checking	Not supported -- must call save() explicitly	Supported -- changes detected automatically
Query language	SQL (via @Query) or derived methods	JPQL or SQL (via @Query with nativeQuery)
Performance predictability	High -- SQL is explicit	Lower -- lazy loading and dirty checking generate implicit SQL
Learning curve	Low -- minimal new concepts	Higher -- entity lifecycle, JPQL, caching, cascading
Best fit	Simpler domains, performance-sensitive code, explicit SQL preference	Complex domain models, rich object graph navigation, rapid development

Comparison

- Practical rule
 - Start with Spring Data JDBC
 - *It is simpler, more predictable, and sufficient for most use cases*
 - *Move to Spring Data JPA when the domain model is complex enough that managing object relationships explicitly becomes burdensome*
 - Use native queries via `@Query(nativeQuery = true)` in either module to access Oracle-specific SQL features
 - *Hints, partitioning syntax, analytic function that are not expressible in JPQL*

Dimension	Spring Data JDBC	Spring Data JPA
Underlying technology	Spring JdbcTemplate -- no ORM	JPA (Hibernate) -- full ORM
Entity lifecycle	None -- objects are plain Java objects	Managed -- JPA tracks changes to persistent entities
Lazy loading	Not supported -- load the whole aggregate	Supported -- related objects loaded on access
Dirty checking	Not supported -- must call save() explicitly	Supported -- changes detected automatically
Query language	SQL (via @Query) or derived methods	JPQL or SQL (via @Query with nativeQuery)
Performance predictability	High -- SQL is explicit	Lower -- lazy loading and dirty checking generate implicit SQL
Learning curve	Low -- minimal new concepts	Higher -- entity lifecycle, JPQL, caching, cascading
Best fit	Simpler domains, performance-sensitive code, explicit SQL preference	Complex domain models, rich object graph navigation, rapid development

Comparison

- The N+1 query problem in JPA
 - Loading a list of employees where each employee lazily loads its department
 - *Generates one SELECT for the employees plus one SELECT per employee for the department*
 - *With 100 employees this is 101 queries where 1 join would suffice*
 - *Always use JOIN FETCH or entity graphs in JPQL, or set relationships to EAGER only with care and measurement*

Dimension	Spring Data JDBC	Spring Data JPA
Underlying technology	Spring JdbcTemplate -- no ORM	JPA (Hibernate) -- full ORM
Entity lifecycle	None -- objects are plain Java objects	Managed -- JPA tracks changes to persistent entities
Lazy loading	Not supported -- load the whole aggregate	Supported -- related objects loaded on access
Dirty checking	Not supported -- must call save() explicitly	Supported -- changes detected automatically
Query language	SQL (via @Query) or derived methods	JPQL or SQL (via @Query with nativeQuery)
Performance predictability	High -- SQL is explicit	Lower -- lazy loading and dirty checking generate implicit SQL
Learning curve	Low -- minimal new concepts	Higher -- entity lifecycle, JPQL, caching, cascading
Best fit	Simpler domains, performance-sensitive code, explicit SQL preference	Complex domain models, rich object graph navigation, rapid development

- JPQL
 - Java Persistence Query Language
 - Defined as part of the JPA specification
 - Standard way to express queries against JPA-managed entities.
- Differences
 - SQL operates on tables and columns
 - *The physical structures in the database*
 - JPQL operates on entity classes and their fields which the Java objects in the application
 - *The query is written in terms of the domain model, not the database schema*

```
// SQL -- references the physical table and column names
SELECT e.employee_id, e.full_name, e.salary
FROM   employees e
WHERE  e.department_id = 10
AND    e.status = 'ACTIVE'
```

```
// JPQL -- references the Java entity class and field names
SELECT e FROM Employee e
WHERE  e.departmentId = 10
AND    e.status = 'ACTIVE'
```

Transactions

- The transaction boundary belongs in the service layer
 - Repository methods are not transactional by default in Spring Data JDBC
 - In Spring Data JPA
 - *Repository methods run in their own transaction by default, but this is usually too granular*
 - *A business operation typically involves multiple repository calls that must succeed or fail together*
 - The correct pattern is to declare transaction boundaries in the service layer, not the repository layer

```
@Service
public class EmployeeTransferService {

    private final EmployeeRepository employeeRepository;
    private final AuditRepository auditRepository;

    // @Transactional ensures both repository calls succeed or both roll back
    @Transactional
    public void transferEmployee(Long employeeId, Long newDepartmentId) {
        Employee employee = employeeRepository.findById(employeeId)
            .orElseThrow(() -> new EntityNotFoundException("Employee not found: " + employeeId));

        Long previousDeptId = employee.getDepartmentId();
        employee.setDepartmentId(newDepartmentId);
        employeeRepository.save(employee);

        // Both operations are in the same transaction
        // If this insert fails, the employee update is also rolled back
        auditRepository.save(new AuditRecord(
            employeeId, "TRANSFER", previousDeptId, newDepartmentId
        ));
    }
}
```

Transactions

- Service layer
 - Methods with `@Transactional` open a transaction
 - Often calls persistence layer method
- Persistence layer
 - Each persistence method runs in a transaction
 - But we have to integrate the two levels of transactions
 - Done identifying how the persistence layer should or should not integrate with the existing transaction

```
@Service
public class EmployeeTransferService {

    private final EmployeeRepository employeeRepository;
    private final AuditRepository auditRepository;

    // @Transactional ensures both repository calls succeed or both roll back
    @Transactional
    public void transferEmployee(Long employeeId, Long newDepartmentId) {
        Employee employee = employeeRepository.findById(employeeId)
            .orElseThrow(() -> new EntityNotFoundException("Employee not found: " + employeeId));

        Long previousDeptId = employee.getDepartmentId();
        employee.setDepartmentId(newDepartmentId);
        employeeRepository.save(employee);

        // Both operations are in the same transaction
        // If this insert fails, the employee update is also rolled back
        auditRepository.save(new AuditRecord(
            employeeId, "TRANSFER", previousDeptId, newDepartmentId
        ));
    }
}
```


Transactions

- Transaction propagation
 - REQUIRED (the default)
 - *Join an existing transaction if one exists; create a new one if not*
 - REQUIRES_NEW
 - *Always create a new transaction, suspending any existing one*
 - *Used for operations that must commit independently such as audit logging that should survive a rollback of the main transaction*
 - SUPPORTS
 - *Run in a transaction if one exists; run without one if not*
 - NOT_SUPPORTED
 - *Always run without a transaction, suspending any existing one*

```
@Service
public class EmployeeTransferService {

    private final EmployeeRepository employeeRepository;
    private final AuditRepository auditRepository;

    // @Transactional ensures both repository calls succeed or both roll back
    @Transactional
    public void transferEmployee(Long employeeId, Long newDepartmentId) {
        Employee employee = employeeRepository.findById(employeeId)
            .orElseThrow(() -> new EntityNotFoundException("Employee not found: " + employeeId));

        Long previousDeptId = employee.getDepartmentId();
        employee.setDepartmentId(newDepartmentId);
        employeeRepository.save(employee);

        // Both operations are in the same transaction
        // If this insert fails, the employee update is also rolled back
        auditRepository.save(new AuditRecord(
            employeeId, "TRANSFER", previousDeptId, newDepartmentId
        ));
    }
}
```

Transactions

- Example
 - The method from the service layer

```
// The outer service -- owns the main business transaction
@Service
public class EmployeeTransferService {

    private final EmployeeRepository    employeeRepository;
    private final AuditService          auditService;
    private final NotificationService    notificationService;
    private final SalaryReadService      salaryReadService;

    // REQUIRED (the default)
    // A transaction is started here before the method body runs.
    // Any method called from here that also uses REQUIRED will JOIN this
    // transaction rather than start a new one.
    @Transactional
    public void transferEmployee(Long employeeId, Long newDepartmentId) {

        Employee emp = employeeRepository.findById(employeeId).orElseThrow();
        emp.setDepartmentId(newDepartmentId);
        employeeRepository.save(emp);

        // auditService.recordTransfer uses REQUIRES_NEW -- see below
        // It starts its own independent transaction and commits it immediately
        // If this line succeeds but something below fails and rolls back,
        // the audit record is already committed and will not be rolled back with it
        auditService.recordTransfer(employeeId, newDepartmentId);

        // This line deliberately fails to demonstrate rollback behaviour
        // The employee save above will be rolled back
        // The audit record written above will NOT be rolled back -- it already committed
        if (newDepartmentId.equals(99L)) {
            throw new RuntimeException("Department 99 is not accepting transfers");
        }

        // notificationService.sendEmail uses SUPPORTS -- see below
        // It runs inside the current transaction because one exists
        // but it does not require one -- it would run the same way without one
        notificationService.sendEmail(employeeId, newDepartmentId);
    }
}
```

Transactions

- Example
 - Method that requires a new transaction

```
// A service that must commit independently of the caller
@Service
public class AuditService {

    private final AuditRepository auditRepository;

    // REQUIRES_NEW
    // When called from within an existing transaction (like the one in
    // transferEmployee above), Spring suspends that transaction, starts a
    // brand new one for this method, commits it when this method returns,
    // and then resumes the suspended transaction.
    //
    // This means the audit record is committed to the database immediately
    // and permanently -- even if the caller's transaction later rolls back.
    // This is exactly the behaviour needed for audit logging: the fact that
    // a transfer was attempted must be recorded even if the transfer failed.
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void recordTransfer(Long employeeId, Long newDepartmentId) {
        auditRepository.save(
            AuditRecord.builder()
                .employeeId(employeeId)
                .action("TRANSFER_ATTEMPTED")
                .newDepartmentId(newDepartmentId)
                .recordedAt(Instant.now())
                .build()
        );
        // This transaction commits here, independently of the caller
    }
}
```

Transactions

- Example
 - Method that requires can work with optional transaction

```
// A service that works with or without a transaction
@Service
public class NotificationService {

    private final EmailClient emailClient;

    // SUPPORTS
    // If called from within a transaction (as it is here, from transferEmployee),
    // it runs inside that transaction -- though in this case it does not do any
    // database work, so the transaction does not affect its behaviour.
    //
    // If called directly with no active transaction (for example, from a
    // scheduled job or a test), it runs without one rather than creating one.
    //
    // SUPPORTS is appropriate here because sending an email does not need a
    // transaction, but the method should not fail if a caller happens to have
    // one active.
    @Transactional(propagation = Propagation.SUPPORTS)
    public void sendEmail(Long employeeId, Long newDepartmentId) {
        // Email sending does not involve the database at all
        // The propagation setting here is defensive -- it says "I do not need
        // a transaction but I will participate in one if you have started it"
        emailClient.send(
            "Transfer notification for employee " + employeeId,
            "Transferred to department " + newDepartmentId
        );
    }
}
```

Transactions

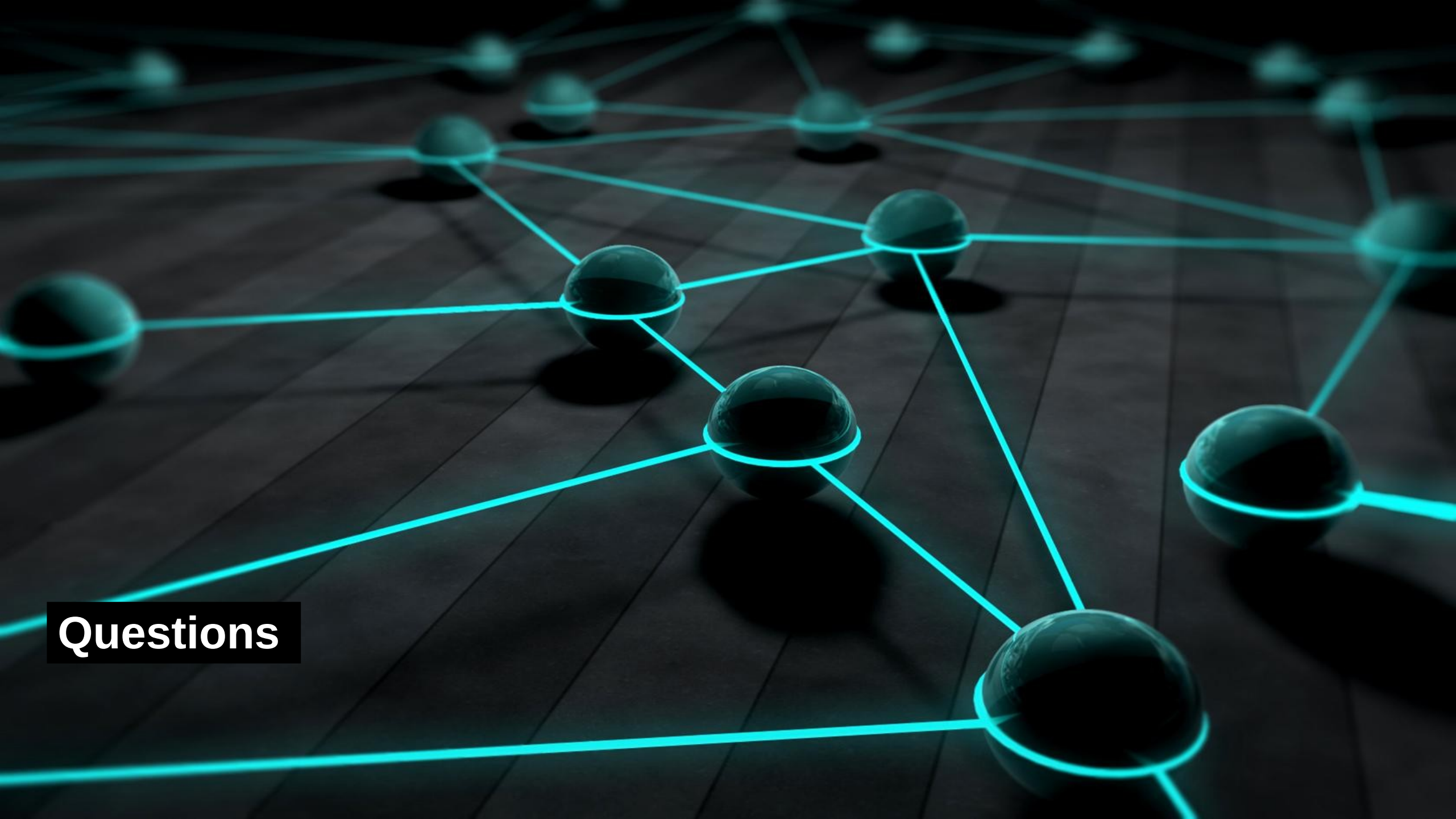
- Common mistakes
 - Call repository methods directly from a controller
 - Rely on the repository's own transaction boundary for operations that span multiple repositories
 - *Then repository call is its own transaction*
 - *If the second call fails, the first has already committed and cannot be rolled back*
 - *The business operation is left in an inconsistent state*

```
// Hint to the database and JPA that no writes will occur
// Allows JPA to skip dirty checking; allows the database to optimize read lock
@Transactional(readOnly = true)
public List<Employee> findActiveEmployees() {
    return employeeRepository.findByStatus("ACTIVE");
}
```

Transactions

- `@Transactional` annotation on the service method
 - Ensures that all repository calls within that method share a single transaction
 - Spring wraps the method call, begins a transaction before the method body executes
 - *Commits it when the method returns normally*
 - *Or rolls it back if an unchecked exception propagates out*
 - `readOnly` flag can avoid acquiring row-level locks that a read-write transaction acquires by default

```
// Hint to the database and JPA that no writes will occur
// Allows JPA to skip dirty checking; allows the database to optimize read lock
@Transactional(readOnly = true)
public List<Employee> findActiveEmployees() {
    return employeeRepository.findByStatus("ACTIVE");
}
```

Questions